Increasing Triplet Subsequence

Company: Microsoft
Round: Offline
Year: 2022
Difficulty: Medium
Topic: Array, Greedy
Source: https://www.desiqna.in/11360/microsoft-interview-experiences-sde-2-2022-set-45

Problem Description

Given an integer array nums, check whether there are **three elements** that appear in increasing order.
We need to find three indices:
i < j < k
such that:
nums[i] < nums[j] < nums[k]
If such three elements exist, return true. Otherwise, return false.
Example
nums = [1, 2, 3, 4, 5]
We can choose:
1 < 2 < 3
and their indices are in increasing order.
So the answer is:
true

Input Format

- The first line contains an integer N, representing the size of the array.
- The second line contains N integers representing the array elements.

Output Format

Print:
true
if an increasing triplet exists,
otherwise print:
false

Sample Input

5
1 2 3 4 5

Sample Output

True

Explanation

Given the array:
[1, 2, 3, 4, 5]
We need to find **three numbers in increasing order** whose positions also come in increasing order.

Here we can choose:

1 < 2 < 3
Their indices are:
0 < 1 < 2
So the conditions i < j < k and nums[i] < nums[j] < nums[k] are satisfied.
Therefore, the output is:
true
In simple terms, **we just need to find any three numbers that appear from left to right in strictly increasing order.**

Approach to Solve This Problem:

1. Keep two variables:
   - first → the smallest number found so far.
   - second → the smallest number greater than first.
2. Traverse the array from left to right.
3. If the current number is smaller than or equal to first, update first.
4. Otherwise, if it is smaller than or equal to second, update second.
5. If the current number is greater than second, we have found:
   first < second < current
   So return true.
6. If we finish the array without finding such a number, return false.

Java Solution:

```java
import java.util.*;
public class Main {
    public static boolean increasingTriplet(int[] nums) {
        int first = Integer.MAX_VALUE;
        int second = Integer.MAX_VALUE;
        for (int num : nums) {
            if (num <= first) {
                first = num;
            }
            else if (num <= second) {
                second = num;
            }
            else {
                return true;
            }
        }
        return false;
    }
```

```
18      }
19      public static void main(String[] args) {
20          Scanner sc = new Scanner(System.in);
21          int n = sc.nextInt();
22          int[] nums = new int[n];
23          for (int i = 0; i < n; i++) {
24              nums[i] = sc.nextInt();
25          }
26          System.out.println(increasingTriplet(nums));
27          sc.close();
28      }
29  }
```

Solution:

The solution uses a **greedy approach** with two variables: first and second.

1. Start with first and second as very large values.
2. Traverse the array from left to right.
3. If the current number is smaller than or equal to first, update first.
4. Otherwise, if it is smaller than or equal to second, update second.
5. If the current number is greater than second, we have found:
   first < second < current
   So an increasing triplet exists and we return true.
6. If the entire array is checked without finding such a number, return false.

Complexity Analysis:

Time Complexity: O(N)

We traverse the array only once.

Space Complexity: O(1)

We use only two variables, first and second, regardless of the input size.